

Boosting II

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026



THOMPSON RIVERS UNIVERSITY

Topics

- 2 Recap
- 3 Introduction
- 4 Stochastic GBMs
- 5 XGBoost
- 6 Variable Importance
- 7 Conclusions
- 8 Coming Up
- 9 Exercises and References

Recap

- Last time we covered **Boosting I**:
 - Procedure.
 - Gradient decent.
 - Hyperparameter tuning.

Introduction

- **Boosting** is an ensemble method that improves accuracy by training weak learners sequentially, with each new model attempting to correct the errors of the previous one.
- We covered gradient boosting machines and tuning hyperparameters.
- We may also borrow from random forests to improve prediction accuracy.
 - Random subsamples and columns from the training data.

Recall: Stochastic Gradient Decent

- An important parameter in gradient descent is the size of the steps which is controlled by the *learning rate*.
 - If the learning rate is too small, then the algorithm will take many iterations to find the minimum.
 - If the learning rate is too high, you might jump across the minimum and end up further away than when you started.
- Not all loss functions are nicely convex!
 - There may be local minima, plateaus, and other irregular terrain making finding a true minimum difficult.
- **Stochastic gradient descent** samples a fraction of the training observations (typically without replacement) and grows the next tree using that subsample.
 - Will not find a *true* minimum, but can help the algorithm jump out of local minima or off of plateaus.

Stochastic GBMs

- **Stochastic gradient boosting machines** introduces randomness into the learning process in order to reduce the chances of getting stuck in local minima and plateaus of loss functions.
- This process leads to additional hyperparameters.
 - 1 Subsample rows before creating each tree.
 - 2 Subsample columns before creating each tree.
 - 3 Subsample columns before considering each split in each tree.

Stochastic GBM Hyperparameters

- Subsampling of rows, such as selecting only 50% or less of the training data, has shown to be beneficial.
 - Typical range: 0.5–0.8
- If there is strong multicollinearity or a lot of noisy variables, subsampling columns can be useful.
- When there are many relevant predictors, a lower values of column subsampling tends to perform well.
 - Similar to m in random forests; if there are many irrelevant variables, you should keep many columns at each split.
- **May tune these hyperparameters at the same time as the tree hyperparameters or after.**

Implementation

- We can use the h2o package in R for implementation.
 - `algorithm = "gbm"`
- The process is computationally demanding and it is doubtful that you will find the true minimum.
- *You can do a grid search of all of hyperparameters or sequentially process batches.*

Illustrative Example 1

- ① Load the *iris* dataset.
- ② Using the hyperparameters that you found trying to predict the Petal.Length in Part I perform a grid search on the stochastic GBM hyperparameters.
 - *Just run the code.*
- ③ Use the final model to predict on the testing set.
- ④ What do you notice?

Example 1

- Import the *Wages.csv* file to conduct the following analysis:
 - ① Run the provided code to perform a 75/25 split.
 - ② Adjust the code from Illustrative Example 1 to construct a Stochastic GBM model with Salary as the response.
 - **Use the hyperparameters that you already found in Part I.**
 - ③ Use the final model to predict on the testing set.
- What do you notice?

XGBoost

- **Extreme gradient boosting** (XGBoost) is an optimized distributed gradient boosting library that is designed to be efficient, flexible, and portable across multiple languages.
- This library provides similar boosting and tree-based hyperparameter options.
- It offers additional hyperparameters and features:
 - Regularization (γ)
 - Parallel processing.
 - Different loss functions and base learners.
 - Save and continue with an existing model.

Regularization

- There are three primary regularization parameters that help to reduce overfitting.
 - ① **gamma**: Pseudo-regularization hyperparameter known as a Lagrangian multiplier $[0 - \infty)$.
 - Specifies a minimum loss reduction required to make a further partition on a leaf node of the tree (prunes the tree).
 - Typically values from 1 - 20 will work.
 - ② **alpha**: Provides an L_1 regularization $[0 - \infty)$.
 - ③ **lambda**: Provides an L_2 regularization $[0 - \infty)$.
 - *If α and λ are both non-zero then the algorithm uses elastic net regularization.*

Dropout

- **Dropout** is an approach to reduce overfitting typically applied in deep learning.
 - Later trees typically improve the prediction for only a small subset of the feature space.
- The idea is to randomly drop trees during the boosting sequence.
- The process is often called Dropout Additive Regression Trees (DART).
 - **The percentage of dropouts is another hyperparameter.**
 - **Typically when regularization does not work, exploring DART is a good option.**
- Arguments for application in R: `booster = "dart", rate_drop = [0.0 - 1.0]`.

Extreme Gradient Boosting Tuning Strategy

- Building upon our stochastic GBMs:
 - 1 Start with a high number of trees and tune learning rate with early stopping.
 - 2 Tune tree-specific hyperparameters.
 - 3 Explore stochastic GBM attributes.
 - 4 If substantial overfitting occurs (e.g., large differences between train and CV error) explore regularization hyperparameters.
 - 5 If you find hyperparameter values that are substantially different from default settings, be sure to retune the learning rate.
 - 6 Obtain final the *best* model.

Illustrative Example 2

- Run the example code to perform a grid search of different regularization values to our *iris* dataset.
 - *We are keeping the hyperparameters that we already found.*
- Note: *To perform cross validation we need to employ one-hot encoding for our categorical variables.*

Variable Importance

- The most commonly used measures of variable importance are:
 - **Gain:** The expected information gain by using a specific variable for a split.
 - In R `vip(model,type = "gain")`
 - **Coverage:** Quantifies the relative number of observations influenced by this variable (how many observations this variable decides the leaf node(s) for).
 - In R `vip(model,type = "cover")`
 - **Frequency:** The percentage representing the relative number of times a particular variable occurs in the trees of the model.
 - In R `vip(model,type = "frequency")`

Example 2

- Determine the variable importance of your model in Example 1.
- Are the same features selected?

Final Thoughts

- GBMs can be very accurate prediction models.
 - Hyperparameter tuning can be very computationally demanding.
- **You need to change the objective if you have a classification problem.**
 - *h2o and xgboost packages.*
- We may be able to make further improvements as our grid searches do not cover all possibilities.
 - You may narrow your grid based on previous iterations.

Next Lecture

- We will discuss **Resampling**.
- Class imbalances.
- How to properly resample.

Exercise 1

- Use the hyperparameters that you have already found for the *Wages.csv* dataset and see if any regularization terms will help.
- Perform a grid search of different regularization terms.
- Does regularization improve your model?
- Save your *best* model.
- How well does this generalize to the testing data?
- Which variables are most important in your dataset?

References & Resources

- 1 Bakker, J. D. (2026) *Applied Multivariate Statistics in R*
 - 2 Peng R. D. (2020) *Exploratory Data Analysis with R*
 - 3 Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction*. Chapman and Hall/CRC
 - 4 Bradley Boehmke B., and Greenwell B. (2020) *Hands-On Machine Learning with R*. Taylor & Francis
-
- caret
 - gbm
 - gbm()
 - xgboost
 - h2o
 - vip()